

Generate more than words.

Understand the path from noise.

Explore how models create images and other data,
then trace a tiny generative process.

🕒 20-25 min core reading · 20-30 min practice · 12 short sections

Your movie assistant can recommend the fictional film *Moonlight Map*. Now imagine creating several poster concepts for it: cool colors and empty space for a calm mood, or warm colors and movement for adventure. There is no single correct picture. A generative model learns patterns that let it produce different plausible samples.

WHAT YOU WILL UNDERSTAND

The main question: how can a learned model turn randomness into structured data?

The practical path: compare generative families, follow noise through a denoising calculation, and explore conditioning and sampling.

Reading path: twelve short sections, three diagrams, two numerical experiments, and a Python workbook. Optional notes hold the extra mathematics.

1. Learn a distribution, not one correct output

A **discriminative** model might predict a poster's genre from its pixels: $p(\text{genre} \mid \text{image})$. A **generative** model describes how data can vary and produces samples from a learned distribution, such as $p_\theta(\text{image})$ or $p_\theta(\text{image} \mid \text{prompt})$. The parameters θ capture learned regularities; a random draw supplies variation.

Training adjusts those parameters using examples and an objective. **Sampling**, also called generation, uses the resulting model to produce an output. A new seed normally changes the sampled variation without retraining the model.

For our movie poster, the prompt is a **condition**: "quiet seaside scene, cool colors, plenty of empty space." Conditions can also be class labels, another image, audio, or layout constraints. A condition steers a distribution; it is not a guarantee that every detail will be obeyed.

Sampling is different from retrieving a stored picture, although models can memorize or reproduce training material. Neither novelty nor originality follows automatically from calling an output "generated."

2. Choose what the model represents

The same output can be represented in different ways. A pixel array is direct but large; a compressed **latent** representation can be smaller. A latent coordinate is not automatically a human-readable concept such as "calmness."

Output	Possible representation	What must stay consistent?
Movie poster	Pixels, patches, or compressed image codes	Objects, layout, colors, and any lettering
Trailer soundtrack	Waveform samples, spectrograms, or audio codec tokens	Rhythm, timbre, and continuity
Trailer video	Frames or compressed spatial and temporal features	Identity and motion across time
Other data	Tabular rows, 3D points, meshes, or discrete symbols	Valid structure and domain constraints

Representation, architecture, objective, and sampler are separate choices. A transformer is an architecture; it can predict the next token or participate in a denoising system. "Diffusion" names a modeling approach, not a requirement to use one particular network architecture.

Compression trades detail against cost. A decoder cannot reliably restore information its representation has discarded. Evaluate the final decoded output, not just a loss measured in latent space.

3. Variational autoencoders: learn a space to sample

A **variational autoencoder (VAE)** pairs an encoder with a decoder. The encoder maps an example to a distribution over latent variables, often described by a mean μ and standard deviation σ . It samples $z = \mu + \sigma \odot \epsilon$, with $\epsilon \sim \mathcal{N}(0, I)$, then the decoder reconstructs the example.

Training balances reconstruction with a penalty encouraging the encoded distribution to resemble a chosen prior. For a new poster, sample z from that prior and decode it; no input poster is required. Reconstruction starts from an existing example; generation starts from a prior draw. See [Auto-Encoding Variational Bayes](#).

➤ One decoder. Two different starting points.

RECONSTRUCTION



Training balances reconstruction with a prior-matching penalty.

NEW GENERATION



Figure 1. Reconstruction encodes an existing poster and samples its latent distribution. New generation samples the prior and uses the learned decoder. The prior path needs no input poster.

▼ Go deeper: the VAE objective and reparameterization

A standard negative evidence lower bound is

$$L = \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x|z)] + D_{KL}(q_\phi(z|x)||p(z)).$$

The first term rewards explaining the example; KL regularizes the latent distribution. Reparameterization separates the random draw from differentiable encoder outputs, letting gradients reach the encoder. Implementations often predict log variance for numerical convenience. Strong reconstruction pressure and strong regularization can compete; low loss does not guarantee sharp or diverse images. An ordinary autoencoder lacks this explicit prior-matching objective, so decoding arbitrary random codes need not produce useful samples.

4. GANs: learn through an adversary

A **generative adversarial network (GAN)** trains two networks together. The generator transforms random noise into synthetic examples. The discriminator learns to distinguish real training examples from generated ones. Its feedback teaches the generator which statistical differences remain detectable.

For movie posters, a discriminator might initially exploit strange textures or broken shapes. As the generator changes, the discriminator's task also changes. At generation time, the basic GAN uses the generator alone: the discriminator is not a mandatory approval gate. See the [original GAN paper](#).

➤ Train two networks. Sample with one.

GENERATED EXAMPLES DURING TRAINING

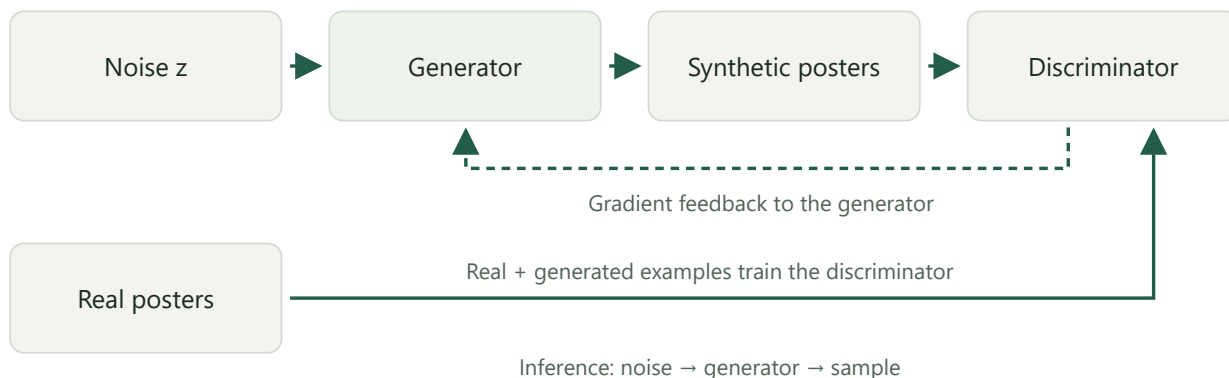


Figure 2. During training, real and generated examples teach the discriminator; feedback through the discriminator teaches the generator. At inference, noise passes through the generator to produce a sample.

▼ Go deeper: mode collapse and unstable training

Mode collapse means the generator covers too little of the real distribution. It might produce attractive posters that all share almost the same composition, even when the examples contain many styles. Sharpness alone would miss this failure.

Because the networks learn against changing opponents, their losses need not improve smoothly together. The idealized equilibrium in the original formulation is not a promise that finite networks and an optimizer will reach it. Practical objectives and regularization vary; evaluate sample variety and quality separately.

5. Autoregressive models: generate in an order

An autoregressive model factors a joint distribution into conditional predictions:

$$p(x_1, \dots, x_n \mid c) = \prod_{i=1}^n p(x_i \mid x_{<i}, c).$$

The elements could be text tokens, image codes, or audio tokens. At sampling time, each new prediction is conditioned on the generated prefix. A poster model might produce a sequence of compressed image codes that a decoder converts into pixels. **PixelCNN** is an early example of autoregressive image modeling at the pixel level.

The ordering makes the probability calculation clear, but sequential generation can add latency and early mistakes can affect later choices. Training often uses known prefixes; generation must live with its own outputs. This is the same teacher-forcing distinction introduced in the language-model chapters.

▼ Go deeper: normalizing flows are another generative family

A **normalizing flow** transforms a simple random variable through invertible mappings: $x = f(z)$. Invertibility and a tractable Jacobian determinant allow a density calculation using change of variables:

$$\log p_X(x) = \log p_Z(f^{-1}(x)) + \log |\det J_{f^{-1}}(x)|.$$

This trades architectural restrictions and computation against tractable density evaluation. Not every decoder is invertible, and not every model with “flow” in its name computes this discrete formula. **Flow matching**, below, is a training method for continuous velocity fields. The two ideas are related but not interchangeable. See [normalizing flows](#) and the [flow-matching paper](#).

6. Diffusion training: learn to denoise

Diffusion adds noise to examples through a known forward process, then learns how to reverse the corruption statistically. In a common noise-prediction formulation, choose a timestep and construct

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

x_0 is clean data. $\bar{\alpha}_t$ is a cumulative signal coefficient between zero and one; smaller values mean stronger corruption. A network receives x_t , the timestep, and optionally a condition. It predicts noise, often using a weighted or simplified mean-squared-error objective against the noise used during training. Other parameterizations predict clean data or a velocity-like target.

The timestep matters: light corruption and nearly pure noise require different estimates. The network learns regularities across examples; it cannot generally identify the exact random noise that corrupted an unseen example. See [Denoising Diffusion Probabilistic Models](#).

↗ Training knows the noise. Generation estimates a path.

TRAINING · UPDATE MODEL



Training examples teach statistical estimates, not exact recovery of every original.

SAMPLING · FIXED MODEL



Figure 3. Training corrupts a known example and compares predicted noise with known noise. Sampling starts from noise and repeatedly uses estimates to update the state. There is no hidden original to recover.

7. Sampling: follow estimates toward data

Generation begins with a noise draw. A **sampler** repeatedly queries the learned denoiser and updates the current state toward a less noisy one. Different samplers use different update rules; some add fresh randomness and some are deterministic given the starting noise. The number of sampling steps is separate from the number of training updates.

Latent diffusion runs this process in a compressed representation, then decodes the final latent to an image. This can reduce computation relative to working on all pixels. It adds a dependency on the encoder and decoder: a good denoising result cannot compensate for every compression failure. See [High-Resolution Image Synthesis with Latent Diffusion Models](#).

Classifier-free guidance (CFG) combines conditional and unconditional predictions. For the noise-prediction convention used here,

$$\hat{\epsilon} = \epsilon_u + s(\epsilon_c - \epsilon_u).$$

$s = 0$ uses the unconditional prediction; $s = 1$ uses the conditional prediction; larger values extrapolate the difference. This requires a model that can provide both predictions, commonly trained by sometimes dropping the condition. Stronger guidance can improve prompt alignment but reduce diversity or introduce artifacts. It is not a universal quality slider. See [Classifier-Free Diffusion Guidance](#).

▼ Go deeper: a denoising calculation and a deterministic sampler

Rearranging the corruption equation gives $\hat{x}_0 = (x_t - \sqrt{1-a}\hat{\epsilon})/\sqrt{a}$, where $a = \bar{\alpha}_t$. If $a = 0.25$, $x_0 = 1$, and $\epsilon = 2$, then $x_t = 2.2321$. Using the true noise recovers 1. Using 1.5 instead gives approximately 1.8660. Noise-estimation error is multiplied by $\sqrt{(1-a)/a} = 1.7321$ here; at high corruption the original is especially uncertain.

A deterministic DDIM-style step toward a larger coefficient a' is

$$x_{\text{next}} = \sqrt{a'}\hat{x}_0 + \sqrt{1-a'}\hat{\epsilon}.$$

This is the zero-additional-noise case, not the universal diffusion update. Our lab uses this form with an analytic estimator. Finite steps approximate a trajectory; a single clean estimate is not the whole sampling process. See [Denoising Diffusion Implicit Models](#). Seeds support repeatable comparisons within a fixed implementation; they do not promise identical outputs across models, devices, or software versions.

8. Flow matching: learn a velocity field

Imagine a movie-poster sample as a point moving from noise toward a data distribution. **Flow matching** learns a vector field that tells each point how to move at each time. For one simple training path, pair noise z with an example x :

$$x_t = (1 - t)z + tx, \quad v_{\text{target}} = x - z.$$

Here $t = 0$ means noise and $t = 1$ means data: **the opposite direction to the diffusion corruption index above**. Many pairs can pass through similar intermediate positions. The learned field estimates the velocity given the current point, time, and condition; it does not receive a secret destination during generation.

To sample, start from noise and solve $dx/dt = v_\theta(x, t)$ numerically. A basic Euler step is $x_{t+\Delta t} = x_t + \Delta t v_\theta(x_t, t)$. The solver, step size, and learned field determine the approximation. A straight training path does not guarantee each sampled trajectory is straight or that one step is sufficient. See [Flow Matching for Generative Modeling](#).

9. Evaluate what the samples actually do

Do not compare generative systems with one attractive poster. Fix a prompt set, sample multiple seeds, and inspect both common and difficult cases.

Question	Movie-poster check
Fidelity	Are shapes and textures coherent?
Diversity and coverage	Do seeds and prompts produce meaningfully different layouts and styles?
Conditioning	Does the calm prompt produce the requested mood and composition?
Exact constraints	Is the title spelled correctly? Are requested objects present in the right number?
Stability	Does editing one detail accidentally change the rest? For video, do identity and motion persist?
Practical cost	What are latency, memory use, and failure rate at the chosen settings?

Distribution-level metrics, similarity scores, and human judgments answer different questions. A single score can hide poor prompt following or missing styles. A lower denoising loss is not itself proof of better lettering or more useful designs.

Use representative data you are entitled to use; track its provenance and avoid exposing private material. For an actual person or voice, obtain the relevant consent. Realism does not establish that a depicted scene happened. Our poster examples are fictional design exercises, not evidence about real films or events.

10. Explore the mechanism, then practice

CORRUPT. ESTIMATE. SAMPLE.

Two coordinates · no image model

Add noise. Estimate. Generate.

Explore an exactly known toy distribution of poster styles. These coordinates illustrate the mathematics; they do not generate pictures or measure quality.

Worked scalar example: clean value $x_0 = 1$, true noise $\varepsilon = 2$, and signal coefficient $a = 0.25$. Signal scale = 0.5000; noise scale ≈ 0.8660 . The corrupted value is 2.2321.

Noise estimate	Clean estimate	Interpretation
2.0000	1.0000	Oracle: uses the true noise
1.5000	1.8660	Noise error changes reconstruction

Noise-error amplification = $\sqrt{((1-a)/a)} = 1.7321$. During generation there is no original value to recover. A sampler starts with noise and follows repeated predictions. With our guidance convention, $s = 0$ ignores the condition, $s = 1$ uses it directly, and $s > 1$ extrapolates. None of these values proves visual quality.

The lab represents posters as **two numbers**, not pixels. It uses two known Gaussian style distributions, not a trained neural network. This makes the calculations inspectable while keeping the limitations visible.

▼ Go deeper: the analytic estimator and the toy distribution

Calm samples have mean $[-1.25, -0.75]$; adventure samples have mean $[1.25, 0.75]$. Both have per-coordinate variance $v_0 = 0.25$. The unconditional distribution is an equal mixture. For a known style mean μ and corrupted point y , elementary Gaussian conditioning gives

$$\mathbb{E}[x_0 \mid y, c] = \mu + \frac{\sqrt{a} v_0}{a v_0 + 1 - a} (y - \sqrt{a} \mu).$$

The corresponding noise estimate is $(y - \sqrt{a} \mathbb{E}[x_0 \mid y, c]) / \sqrt{1 - a}$. Without the style label, the lab averages the two posterior means using their probabilities given y . A posterior mean minimizes expected squared error; it need not equal the original point or look like a typical sample.

Generation starts from a standard-normal draw at $a = 0.001$ and follows a finite cosine-spaced schedule to $a = 1$. That starting distribution only approximates the toy forward process's terminal distribution. Even this exact analytic denoiser plus a finite sampler does not promise exact draws from the target distribution. No coordinate here measures visual quality.

Exercise 1: noise versus uncertainty. Keep Calm and seed 42. Compare noise levels 1 and 7 with the conditional estimator. Then choose the teaching oracle. Explain why the oracle can reconstruct the original while the ordinary estimator usually cannot. Do not conclude that one sample's error must increase monotonically at every level.

▼ Check your reasoning

The oracle has the exact noise used in corruption, so rearranging the equation recovers the original. The estimator knows a distribution and the observed point; several originals could plausibly explain it. Its mean estimate summarizes that uncertainty.

Exercise 2: conditioning versus guidance. In the generation panel, use seed 42 and 8 steps. With guidance 0, switch between Calm and Adventure: the path should stay the same. Repeat at guidance 1, then 7. Explain why moving farther toward a condition is not evidence of better image quality.

▼ Check your reasoning

At zero, the conditional term contributes nothing. At one, it replaces the unconditional prediction. At seven, the difference is extrapolated. A large coordinate change shows stronger numerical steering, not improved aesthetics, diversity, or constraint satisfaction.

Exercise 3: sampling versus reconstruction. Download the workbook and run the commands below. Compare 4 and 16 sampling steps using the same seed. Check that the history changes, while the starting noise does not. Name one reason why more steps cannot repair a bad learned distribution.

▼ Check your reasoning

A finer solver can follow the same learned field more accurately. If that field misses a style or learned the wrong associations, improving the numerical approximation still follows the wrong model. The seed fixes the starting draw, not the entire sampling algorithm.

The four-file workbook runs on Python 3.10+ using only the standard library. Extract the ZIP, then run these commands from its folder.

```
python generation.py
python generation.py --level 7 --estimator oracle
python generation.py --style adventure --guidance 3 --steps 16 --seed 7
python -m unittest -v test_generation.py
```

11. Check your understanding

01 How does basic VAE generation differ from reconstruction?

- A. Generation samples a latent from a prior and decodes it; reconstruction encodes an existing input first.
- B. Generation requires selecting an original training image to copy.
- C. Reconstruction and generation always start from the same input image.

Answer: A. The encoder helps learn and reconstruct, while a new prior draw can be sent directly to the decoder. A useful prior and decoder are learned together through the VAE objective.

02 A GAN produces sharp posters with almost identical layouts. What should you investigate?

- A. Whether sharpness guarantees perfect coverage.
- B. Mode collapse or poor coverage of the target distribution.
- C. Whether every generation needs to pass a discriminator approval gate.

Answer: B. Sample fidelity and variety are distinct. A generator can create convincing individual outputs while missing much of the diversity in the data.

03 What is the noise target during common diffusion training?

- A. Noise secretly saved for every future generated image.
- B. The exact next poster the user will request.
- C. The random noise used to corrupt a training example at a chosen timestep.

Answer: C. Training has access to the clean example and the sampled noise. At generation time there is no original poster to recover; the learned estimator guides a sampling process.

04 In $\hat{\epsilon} = \epsilon u + s(\epsilon c - \epsilon u)$, what does $s = 1$ mean?

- A. Use only the unconditional prediction.
- B. Use the conditional prediction without extrapolating beyond it.
- C. Guarantee perfect prompt following and maximum diversity.

Answer: B. At zero the formula gives ϵu ; at one it gives ϵc . Larger values extrapolate. Different guidance conventions exist, so always inspect the actual formula.

05 What does a flow-matching sampler do?

- A. Look up the secret final image assigned to every seed.
- B. Assume one Euler step is always exact.
- C. Integrate a learned time-dependent velocity field starting from noise.

Answer: C. Training paths provide velocity targets. Generation follows the learned field without a known destination, and numerical solvers introduce approximation error.

12. Connect generation to multimodal systems

You can now distinguish reconstructing an input, predicting a sequence, transforming a latent variable, denoising a sample, and integrating a velocity field. These are different mechanisms for modeling and sampling distributions; they are not a ladder in which one family universally replaces every other.

Return to the movie assistant: a recommender ranks films, a retrieval system finds evidence, and a generative model might create a poster concept. A **multimodal system** can connect these roles across text, images, documents, audio, and video. Chapter 30 explains those connections, including what it means to understand an image versus generate one.

▼ Quick-reference glossary

Term	Meaning
Generative distribution	A model of how examples can vary, from which samples can be produced.
Condition	Information used to steer generation, such as text, a label, or an image.
Latent variable	A hidden representation or random variable used to model data.
Prior	A starting probability distribution, often used to sample latent variables.
Posterior	A distribution after incorporating observed evidence.
VAE	A latent-variable model trained using reconstruction and variational regularization.
GAN	A generator trained using feedback from a learned discriminator.
Mode collapse	Producing too little of the variety present in the target distribution.
Autoregressive	Predicting each element conditioned on earlier elements.
Normalizing flow	An invertible transformation connecting distributions with density tracking.
Diffusion	Learning generation through the statistical reversal of a noise process.
Denoiser	A model estimating clean data, noise, or a related target from a noisy state.
Sampler	The procedure turning a model and randomness into a generated output.
CFG	Combining conditional and unconditional predictions to steer generation.
Flow matching	Training a velocity field along paths between distributions.
Seed	An initializer for pseudorandom draws within an implementation.